

Design Patterns

Behavioral Patterns

These slides were redrawn from the material of **Alexander Abramovich**. Behavioral patterns are concerned with algorithms and the assignment of responsibilities between objects — they describe not just patterns of objects or classes but also the patterns of communication between them.



Meeting Outline

This session covers five essential **behavioral design patterns**. Each pattern addresses a distinct challenge in object collaboration and communication.

1

Observer

One-to-many dependency notification

2

Template Method

Algorithm skeleton with deferred steps

3

Strategy

Interchangeable algorithm families

4

Iterator

Sequential access to collections

5

Command

Encapsulate requests as objects



Learning Outcomes

Pattern Familiarity

Gain a solid understanding of five behavioral design patterns — Observer, Template Method, Strategy, Iterator, and Command — along with their internal structures and participant roles.

Pattern Selection

Develop the ability to identify the right pattern for a given design problem, understanding the trade-offs, benefits, and drawbacks of each approach in real-world software development scenarios.

Main Issue

Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

This problem arises frequently in GUI frameworks, event systems, and reactive architectures. The core challenge is keeping multiple dependent objects in sync with a single source of truth — without tightly coupling them to it. The Observer pattern solves this by establishing a publish/subscribe relationship between a **Subject** and any number of **Observers**.

Example: Real-Time Monitoring

Consider a **Patient monitoring system** in a hospital. A single patient (the Subject) is observed by multiple parties simultaneously. Each observer reacts independently when the patient's state changes.



Operator

Receives alerts and takes action when vital signs cross thresholds.



Monitor

Displays real-time patient vitals on a dedicated screen.



Patient

Acts as the Subject — its state (vitals) drives all notifications.



Log File

Automatically records all state changes for audit and review.

Example: Spreadsheet Application

A classic MVC scenario: the **Data** object is the Subject. Multiple visual representations act as Observers, each updating independently when the underlying data changes — without the data model knowing anything about the views.



Pie Chart

Redraws proportional slices when data values change.



Table

Refreshes rows and cells to reflect updated values.



Histogram

Recalculates bar heights based on new distributions.



Data

The Subject — stores the canonical state and notifies all views.

Structure: Class Diagram

Subject (Abstract)

```
observers : vector
attach(o : Observer)
detach(o : Observer)
notify() {
  for all observers {
    update();
  }
}
```

ConcreteSubject

```
subjectState
attach(o)
detach(o)
notify()
getState()
```

Observer (Abstract)

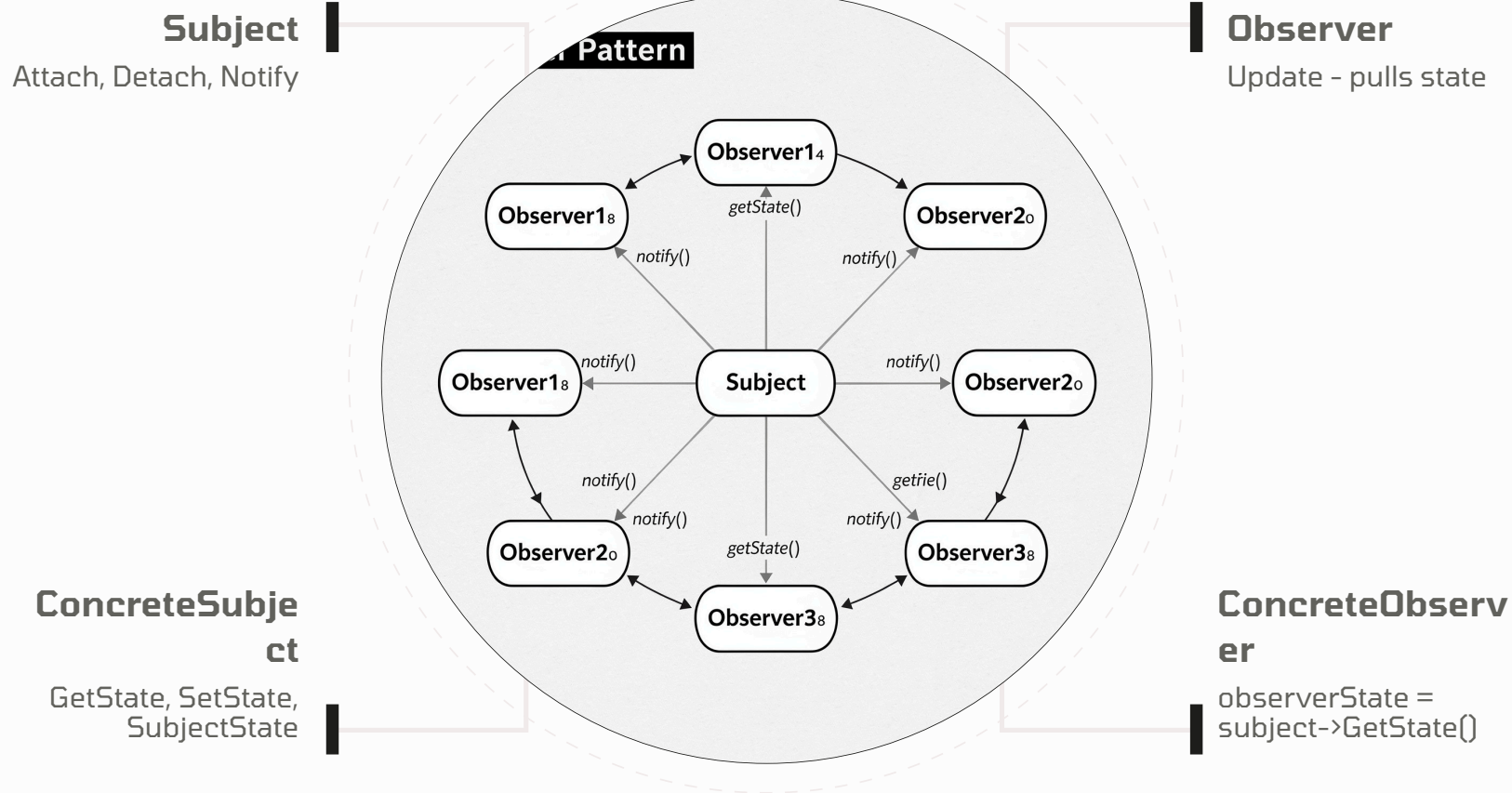
```
update(...)
```

ConcreteObserver

```
subject → reference
observerState
update() {
  observerState =
    subject.getState();
}
```

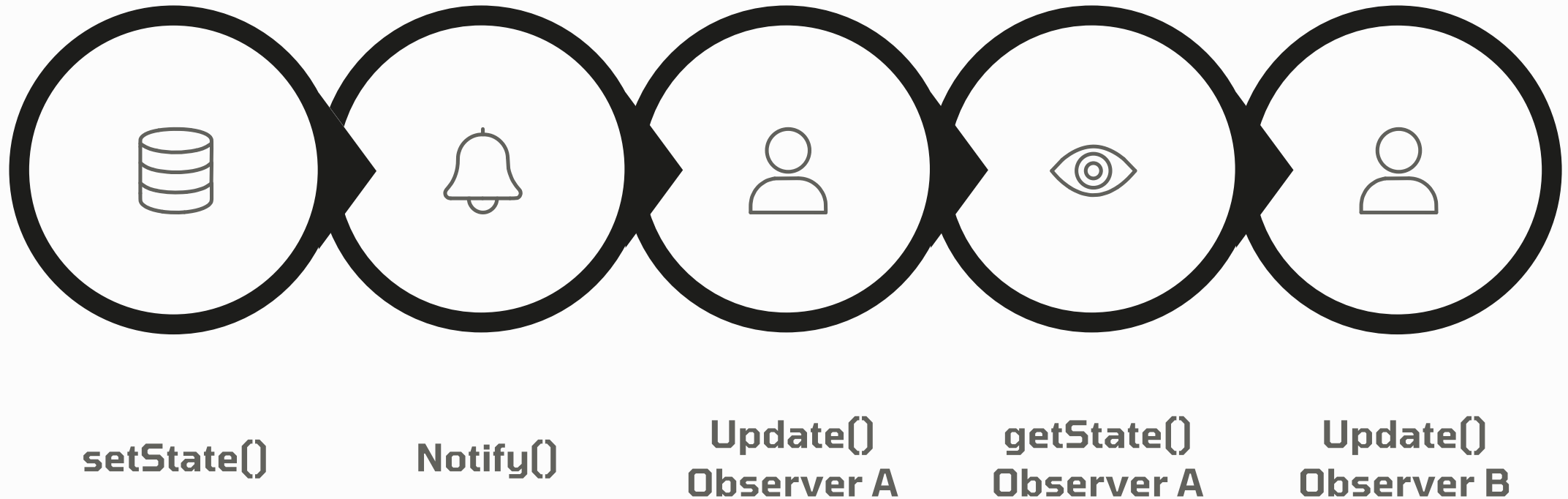
The **ConcreteObserver** maintains a reference to its subject and keeps its local state consistent via `update()`.

Structure: Roles & Responsibilities



The **Subject** maintains a list of **Observers** and calls `Notify()` on state change. Each **Observer** calls `GetState()` on the **Subject** to synchronize its own `observerState`. This decouples the **Subject** from any specific **Observer** implementation.

Sequence Diagram



When `setState()` is called on the Subject, it triggers `Notify()`, which iterates over all registered observers and calls `Update()` on each. Each Observer then pulls the new state via `getState()` — demonstrating the **pull model** of notification.

Participants

Subject

Knows its observers — any number of Observer objects may observe it. Provides an interface for **attaching** and **detaching** Observer objects dynamically.

Observer

Defines an **updating interface** for objects that should be notified of changes in a Subject. Acts as the abstract notification contract.

ConcreteSubject

Stores state of interest to ConcreteObserver objects. **Sends a notification** to its observers whenever its state changes.

ConcreteObserver

Maintains a reference to a ConcreteSubject. Stores state that should stay consistent with the subject's and implements the Observer updating interface.

Benefits



Abstract Coupling

The Subject holds only a reference to the abstract Observer interface — it knows nothing about concrete observer classes. This means Subject and Observer can vary and be reused independently, making the system highly extensible.



Broadcast Communication

Notifications are sent to **all** registered observers automatically. The Subject doesn't need to specify receivers explicitly — any object implementing the Observer interface can subscribe and receive updates at runtime.

Drawbacks



Unexpected Updates

Because observers have **no knowledge of each other's presence**, they can be blind to the ultimate cost of changing the subject. A seemingly innocuous operation on the Subject may cause a **cascade of updates** to observers and their dependent objects — potentially leading to performance bottlenecks or circular update chains that are difficult to trace and debug.

Implementation-Related Issues

Multiple Subjects per Observer

An observer may observe more than one subject. The `update()` method should accept the Subject as a parameter to distinguish the source of a notification.

Push vs. Pull Models

The **push model** sends detailed change data with the notification; the **pull model** sends a minimal signal and lets observers query for details. Pull is more flexible but requires extra calls.

Observer Deletion Awareness

A Subject must be notified when an Observer is deleted to avoid dangling references. Always detach observers before destroying them.

Interest Specification

Observers can register interest in specific events only, making notifications more targeted and reducing unnecessary update calls.

Meeting Outline

01	02	03
Observer	Template Method	Strategy
✓ Covered	← Up Next	Upcoming
04	05	
Iterator	Command	
Upcoming	Upcoming	

Main Issue

Define the skeleton of an algorithm in an operation, deferring some steps to subclasses. Subclasses can redefine certain steps of an algorithm without changing the algorithm's overall structure.

This pattern is particularly useful when multiple classes share the same **high-level algorithm** but differ in specific implementation details. Rather than duplicating the shared scaffolding, the Template Method centralizes it in an abstract base class, allowing subclasses to **plug in** their specific behavior through well-defined hook methods.

Example: Graphics Editor

Shape (AbstractClass)

```
- x : int
- y : int
- visible : bool
+ getX() : int
+ getY() : int
+ isVisible() : bool
+ setVisible(bool) ← template
+ moveTo(newX, newY) ← template
  calls setX(), setY(),
  setVisible()
```

Circle (ConcreteClass)

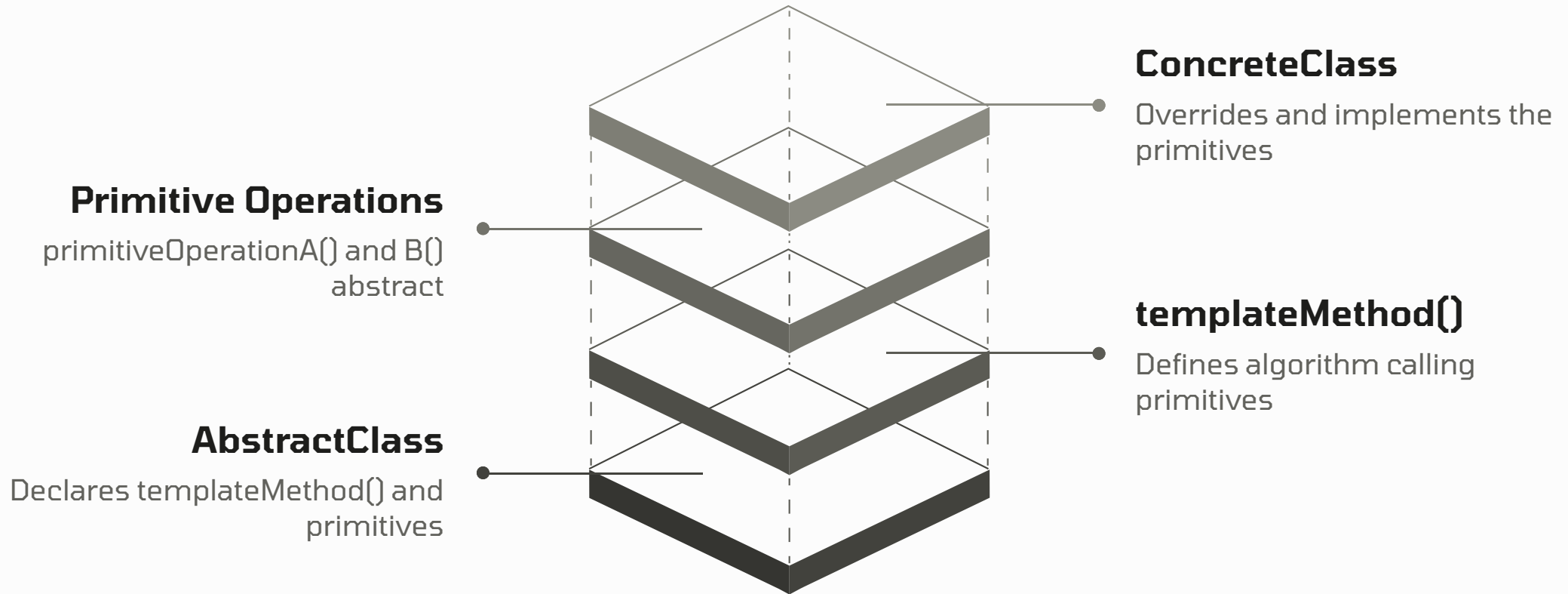
```
- radius : int
+ getRadius() : int
+ setRadius(r : int)
+ setVisible(show : bool) ← overrides
```

Rectangle (ConcreteClass)

```
- width : int
- height : int
+ getWidth(), getHeight()
+ setWidth(), setHeight()
+ setVisible(show : bool) ← overrides
```

Both subclasses override `setVisible()` with shape-specific rendering logic while inheriting `moveTo()` unchanged.

Structure



The `templateMethod()` in **AbstractClass** defines the invariant algorithm skeleton and calls **primitive operations** — abstract methods that must be implemented by **ConcreteClass**. The template method itself should *not* be overridden, only the primitive operations.

Participants

AbstractClass

Defines **abstract primitive operations** that concrete subclasses implement to carry out steps of an algorithm. Implements the `templateMethod()`, which calls primitive operations as well as other operations defined in `AbstractClass` or on other objects. The template method enforces the algorithm's structure.

ConcreteClass

Implements the **primitive operations** to carry out subclass-specific steps of the algorithm. Each `ConcreteClass` can vary its implementation of primitive operations while still reusing the common algorithm skeleton defined in `AbstractClass`.

Benefits

Definition of a Basic Behavior

The template method establishes a **common algorithm skeleton** in the base class, eliminating code duplication across subclasses. Subclasses customize only the parts that need to vary — ensuring consistency while enabling flexibility.

Hollywood Principle

"Don't call us, we'll call you." The base class controls the flow and calls into subclasses at well-defined extension points, giving the framework control over the algorithm's structure while still allowing subclass-level customization.



Implementation-Related Issues

→ Protect the Template Method

The `templateMethod()` itself **should not be overridden** by subclasses. In C++, declare it as `non-virtual` (or `final` in Java/C#) to enforce this constraint and preserve the algorithm's structure.

→ Minimize Primitive Operations

The fewer the abstract methods, the less burden on subclasses. Each required override is a contract — keep the list lean and focused to make subclassing as straightforward as possible.

→ Naming Convention for Hooks

Identify operations that *should* be overridden by adding a distinguishing prefix (e.g., `do` or `hook`) to their names. This makes the extension interface explicit and self-documenting for subclass implementors.

Meeting Outline

01	02	03
Observer	Template Method	Strategy
✓ Covered	✓ Covered	← Up Next
04	05	
Iterator	Command	
Upcoming	Upcoming	

Main Issue

Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from the clients that use it.

When a class needs to support multiple behaviors or algorithms that might differ in performance or functionality, embedding that logic directly leads to complex conditional branches and rigid code. The Strategy pattern addresses this by extracting each algorithm into its own class, making them easily **swappable at runtime** without modifying the client.

Example: GUI Layout Manager

A **Container** holds a collection of UI components. Instead of hardcoding how components are arranged, it delegates layout logic to a **LayoutManager** strategy. Different layout strategies can be swapped without modifying the Container.

LayoutManager (Strategy)

```
+ getLocation(c:Component)
```

Abstract interface implemented by all layout strategies.

Container (Context)

```
components : vector  
+ layoutComponents()
```

Delegates to the current LayoutManager without knowing its type.

BorderLayout

```
+ getLocation(c:Component)
```

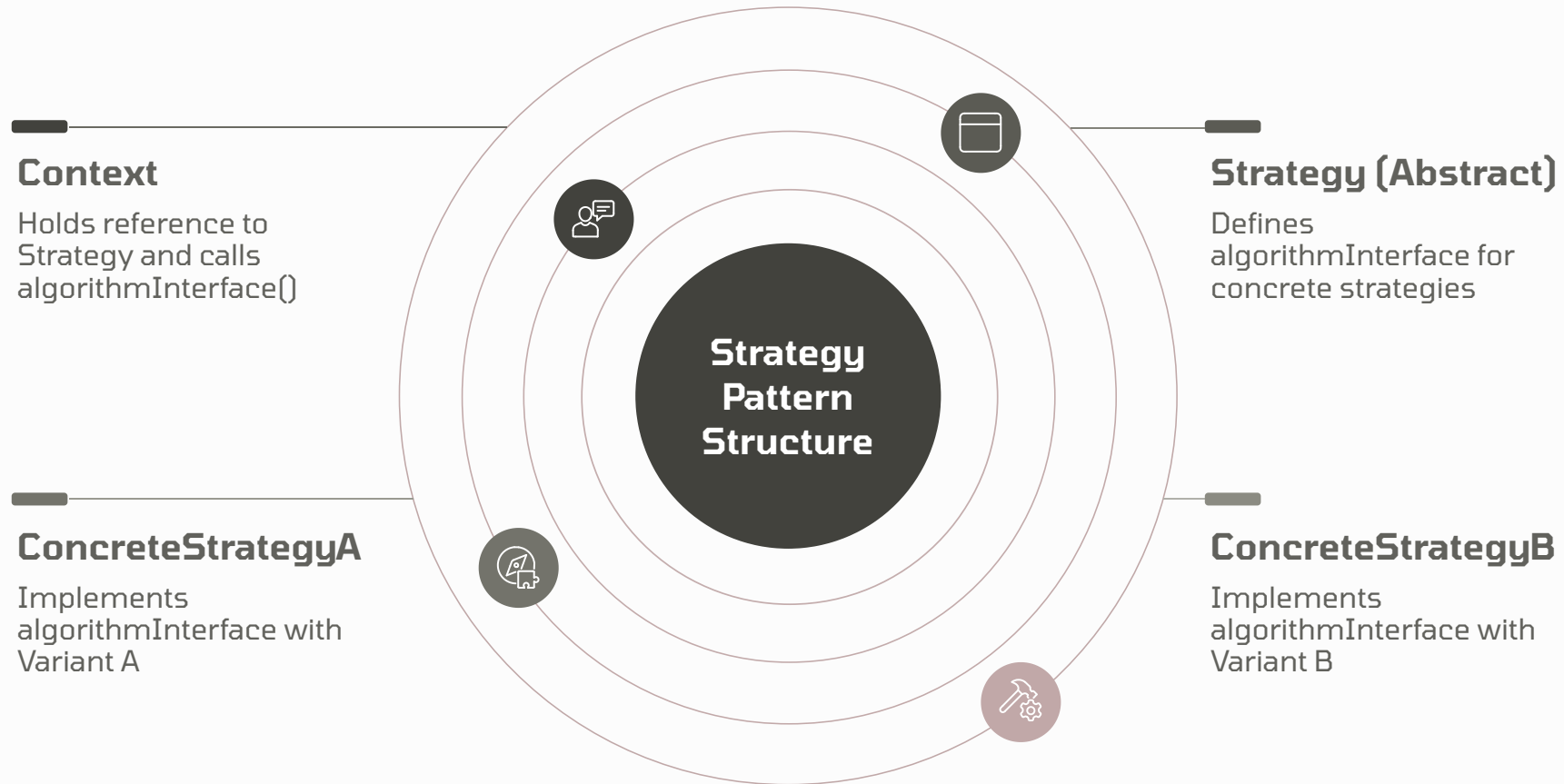
Positions components in North/South/East/West/Center regions.

GridLayout

```
+ getLocation(c:Component)
```

Arranges components in a uniform grid of rows and columns.

Structure



The **Context** is configured with a `ConcreteStrategy` object and calls its `algorithmInterface()` without knowing which concrete implementation is in use. This indirection allows runtime algorithm switching and complete independence between context logic and algorithm logic.

Participants

Strategy

Declares an interface **common to all supported algorithms**. Context uses this interface to call the algorithm defined by a ConcreteStrategy — ensuring full interchangeability.

ConcreteStrategy

Implements the algorithm using the Strategy interface. Each ConcreteStrategy provides a different algorithm or behavior variant, all behind the same interface.

Context

Is configured with a ConcreteStrategy object. Maintains a reference to a Strategy object and may define an interface that lets the Strategy access its data when needed.

Benefits



Algorithm Transparency

The client is **not aware of algorithm implementation details**. It interacts only through the abstract Strategy interface, keeping business logic cleanly separated from algorithmic logic.



Runtime Switching

Implementations can be **switched at run-time** by assigning a different ConcreteStrategy to the Context — enabling dynamic behavior changes without recompilation.



Alternative to Subclassing

Strategy is a **composition-based alternative** to subclassing the Context for different behaviors. It avoids deep inheritance hierarchies by favoring object composition over class extension.

Drawbacks

Client Awareness of Strategies

The client must be **aware of the different Strategy implementations** in order to choose the appropriate one. This exposes algorithmic options to the client, which may introduce coupling to the strategy family — the client needs to understand what distinguishes each strategy before selecting one.

Communication Overhead

There is a **communication overhead** between the Strategy and Context. The Context may pass data to the Strategy that it doesn't actually need, or the Strategy interface may force all ConcreteStrategies to expose methods that only some of them use — bloating the interface.

Implementation-Related Issues

→ Define Interfaces Carefully

The Strategy and Context interfaces must be designed carefully. A well-defined interface passes exactly what strategies need — no more, no less. Consider whether strategies need direct access to Context data or whether Context should package it up before calling the strategy.

→ C++ Templates as Strategies

In C++, **templates** can be used to configure a class with a strategy at compile time — avoiding virtual dispatch overhead. A well-known example is STL's `Allocator` template parameter, which allows customizing memory management without runtime polymorphism.

→ Default Strategy Behavior

Context may define a **default behavior** for the case that no Strategy object is supplied. This makes the Strategy optional and backward compatible — the Context falls back to sensible defaults when no explicit strategy is configured.

Meeting Outline

01	02	03
Observer	Template Method	Strategy
✓ Covered	✓ Covered	✓ Covered
04	05	
Iterator	Command	
← Up Next	Upcoming	

Main Issue

Provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation.

Collections can be implemented in many ways — arrays, linked lists, trees, hash maps — but client code should not need to know these details just to iterate over elements. The Iterator pattern decouples traversal logic from the collection's internal structure, allowing clients to iterate uniformly regardless of how data is stored. This also enables **multiple simultaneous traversals** of the same collection.

Example: Collection Traversal

Consider a playlist in a media player. The underlying storage could be a linked list, array, or tree — but the client (the player UI) just needs to call `next()` to get the next song. Different iterators can even implement different traversal orders (shuffle, alphabetical, chronological) over the same collection.



Aggregate (Collection)

Stores elements internally (array, list, tree).
Exposes only a `createIterator()` factory method — not its storage structure.



Iterator

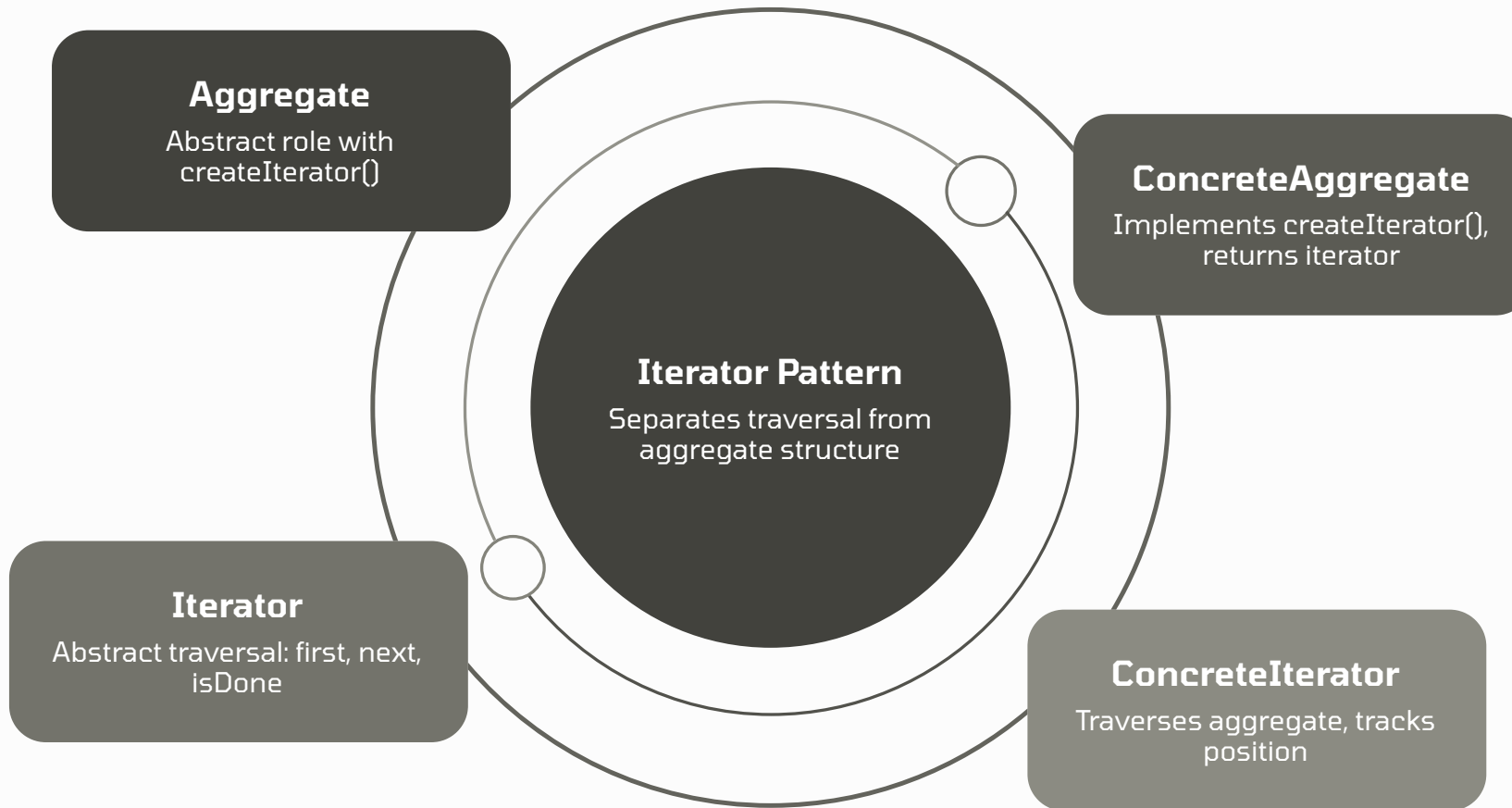
Maintains traversal state (current position).
Provides `first()`, `next()`, `isDone()`, and `currentItem()`.



Client

Uses only the Iterator interface — completely unaware of whether it is traversing a list, tree, or any other data structure.

Structure



ConcreteAggregate implements `createIterator()` by returning a **ConcreteIterator** bound to itself. **ConcreteIterator** tracks position within the **ConcreteAggregate** and implements all **Iterator** traversal methods. Client code depends only on the abstract **Aggregate** and **Iterator** interfaces.

Participants

Iterator

Defines the interface for accessing and traversing elements: `first()`, `next()`, `isDone()`, and `currentItem()`. This abstract interface decouples client traversal code from any concrete collection.

ConcreteIterator

Implements the Iterator interface and keeps track of the current position in the traversal of a ConcreteAggregate. Manages traversal state so multiple iterators can traverse the same collection simultaneously.

Aggregate

Defines an interface for **creating an Iterator** object via `createIterator()`. This is typically a factory method that returns the appropriate ConcreteIterator.

ConcreteAggregate

Implements the `createIterator()` operation to return an instance of the corresponding ConcreteIterator. It stores the actual collection elements in its chosen internal structure.

Benefits



Simplified Interface

The Iterator pattern **simplifies the *Aggregate interface*** — collections no longer need to expose traversal operations directly. The traversal responsibility is cleanly delegated to the Iterator object.



Multiple Simultaneous Traversals

Since each Iterator object maintains its own traversal state, **multiple *traversals*** can be active over the same aggregate at the same time — useful for nested iterations and concurrent access scenarios.



Uniform Traversal Interface

Clients iterate over different types of collections using the **same interface**. Switching the underlying data structure (e.g., from array to tree) does not require changes to client traversal code.

Drawbacks

Robustness During Modification

If the aggregate is **modified while being traversed**, the iterator may become invalid. Ensuring that insertions or deletions do not corrupt an in-progress iteration requires careful implementation — either through copy-on-write semantics, versioning, or fail-fast iterators that throw on concurrent modification.

Added Complexity for Simple Collections

For very simple, fixed-size collections, the overhead of defining a separate Iterator class hierarchy can feel heavyweight. The pattern is most valuable when **collection type or traversal behavior varies** — for trivial cases, direct index-based access may be simpler.

Implementation-Related Issues

→ External vs. Internal Iterators

An **external iterator** (client controls traversal by calling `next()`) gives more flexibility. An **internal iterator** (collection applies an operation to each element) is simpler to use but gives the client less control. External iterators are more common in GoF-style implementations.

→ Robust Iterators

A **robust iterator** ensures that insertions and deletions during traversal do not cause elements to be skipped or visited twice. Implementing this typically requires the iterator to register with the aggregate and be notified of structural changes.

→ Null Iterators

A **NullIterator** (one where `isDone()` always returns true) is useful for handling leaf nodes in composite structures like trees — enabling uniform traversal code without special-casing leaf vs. composite nodes.

Meeting Outline

01	02	03
Observer	Template Method	Strategy
✓ Covered	✓ Covered	✓ Covered
04	05	
Iterator	Command	
✓ Covered	← Up Next	

Main Issue

Encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations.

In many systems, the object that *issues* a request needs to be decoupled from the object that *fulfills* it. Hardcoding method calls creates tight coupling between invokers and receivers. The Command pattern turns requests into **first-class objects** — enabling undo/redo, macro recording, operation queuing, and remote execution, all without the invoker knowing anything about the receiver.

Example: Text Editor with Undo

In a text editor, toolbar buttons and menu items are **Invokers**. Each triggers a Command object that knows how to execute (and undo) a specific operation on the document (the **Receiver**). Commands can be stacked in a history list for multi-level undo.



Invoker (Toolbar Button)

Holds a reference to a Command. Calls `execute()` without knowing what operation will be performed or on which object.



Command (CopyCommand)

Encapsulates the request and its receiver. Implements `execute()` and `undo()` — storing enough state to reverse the operation.



Receiver (Document)

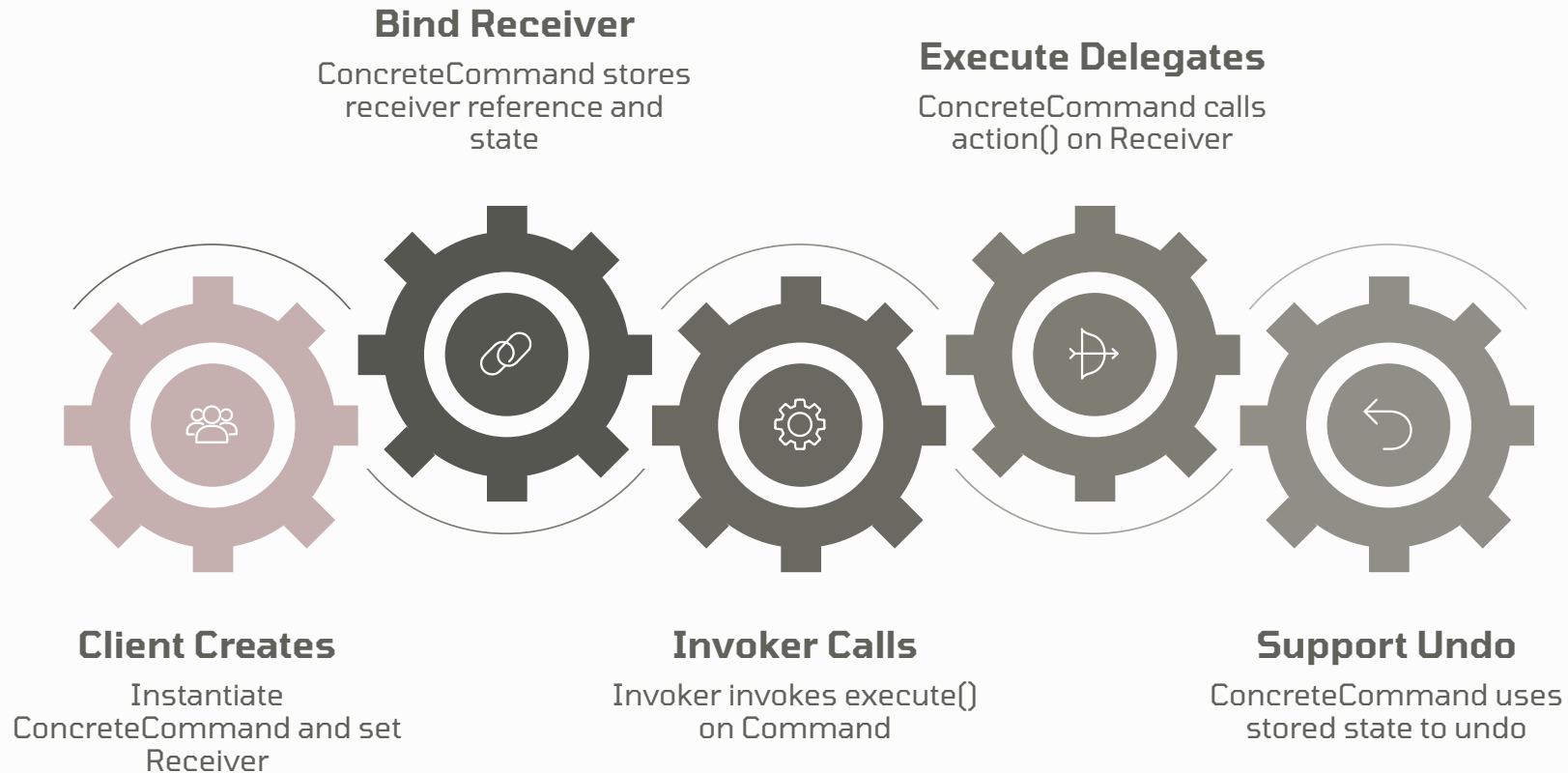
The object that actually performs the work when the command's `execute()` is called — e.g., copying selected text to the clipboard.



Command History

A stack of executed Command objects that enables **multi-level undo/redo** by calling `undo()` in reverse order.

Structure



The **Client** creates a ConcreteCommand and binds it to a Receiver. The **Invoker** stores and triggers the Command via `execute()`. The **ConcreteCommand** translates the call into one or more operations on the Receiver, keeping any state needed to support `undo()`.

Participants

Command

Declares an abstract interface for executing an operation, typically `execute()` and optionally `undo()`. All concrete commands implement this interface, making them interchangeable to the Invoker.

ConcreteCommand

Implements the Command interface by binding together a **Receiver** object and a specific action. Stores any state required to support undoing the operation (e.g., previous text, previous position).

Invoker

Asks the Command to carry out the request. The Invoker is **decoupled from the Receiver** — it does not know which receiver handles the request or how it is handled.

Receiver

Knows how to perform the operations associated with carrying out a request. **Any class can serve as a Receiver** — the Command pattern imposes no constraints on which objects can be Receivers.

Client

Creates a ConcreteCommand object and configures it with its Receiver — assembling the full command object before passing it to the Invoker.

Benefits



Decouples Invoker from Receiver

The Invoker has **no dependency on the Receiver** — it only knows the Command interface. This makes it easy to add new commands without changing existing invoker code.



Undo / Redo Support

Commands can store the **state necessary to reverse** an operation. A command history stack enables arbitrarily deep undo/redo — a critical feature in editors, IDEs, and design tools.



Queuing and Logging

Because commands are objects, they can be **serialized, queued, and logged**. This makes it straightforward to build operation schedulers, transaction logs, and audit trails — or to replay operations after a crash.



Macro Commands

A **MacroCommand** is a composite that holds a list of simpler commands and executes them in sequence. This enables batch operations and scriptable workflows without changing individual command implementations.

Drawbacks

Proliferation of Command Classes

Every distinct operation requires its own ConcreteCommand class. In systems with many operations, this can lead to a **large number of small classes** — increasing the codebase size and maintenance burden. Modern languages mitigate this with lambdas and first-class functions.

Undo Complexity

Implementing `undo()` correctly requires each command to **capture enough state** to fully reverse its effects. For commands with side effects on external systems (file I/O, network calls), reversibility may be impractical or impossible to guarantee.

Implementation-Related Issues

→ How Much Does a Command Do?

At one extreme, a command merely establishes the Receiver-action binding (thin command); at the other, it implements the operation entirely itself (fat command). **Thin commands** favor reuse; **fat commands** are useful when no suitable Receiver exists or when the command logic is too specific to generalize.

→ Avoiding Error Accumulation in Undo

Repeatedly executing and undoing commands (especially those involving floating-point arithmetic or UI state) can lead to **accumulated errors**. Consider storing snapshots of pre-operation state rather than computing reverse operations to ensure precision.

→ Supporting Undo and Redo

Store executed commands in a **history list**. Call `undo()` in reverse order to undo, and `re-execute()` in forward order to redo. Commands must be designed to be idempotent or to fully capture pre-execution state for this to work correctly.

→ MacroCommand Implementation

Implement a `MacroCommand` class that holds a list of Commands and delegates `execute()` and `undo()` to each in sequence (or reverse). MacroCommands should themselves be treated as Commands, enabling nested macros and compositional scripting.

Pattern Comparison

All five behavioral patterns decouple responsibilities — but each targets a distinct kind of coupling. Use this summary to select the right tool for each design problem.

Pattern	Intent	Key Mechanism	Classic Use Case
Observer	Notify dependents on state change	Subject maintains observer list; broadcasts updates	Event systems, MVC views
Template Method	Define algorithm skeleton, defer steps	Abstract base class with hook methods	Frameworks, parsers, rendering pipelines
Strategy	Swap algorithms at runtime	Context delegates to interchangeable Strategy	Sorting, layout, compression algorithms
Iterator	Traverse collections uniformly	Separate traversal state into Iterator object	STL iterators, for-each loops, cursors
Command	Encapsulate requests as objects	Invoker calls execute() on Command; stores history	Undo/redo, queues, macro recording

A warm, soft-focus photograph of a desk. In the foreground, an open notebook with a black pen resting on it sits on a wooden surface. To its right is a white cup of coffee on a saucer. In the background, a silver laptop is partially visible. The scene is lit with warm, golden light, suggesting a cozy workspace.

Key Takeaways

Behavioral Patterns Manage Communication

They address how objects collaborate and distribute responsibility — separating *what* needs to happen from *who* does it and *when*.

Each Pattern Targets a Distinct Problem

Observer for notifications, Template Method for algorithm skeletons, Strategy for interchangeable behaviors, Iterator for uniform traversal, Command for encapsulated requests.

Trade-offs Always Exist

Every pattern introduces indirection and new abstractions. Use them when the flexibility they provide outweighs the added complexity — not as a reflex.